Desafio técnico back-end · Revelatio Studio

Task Manager

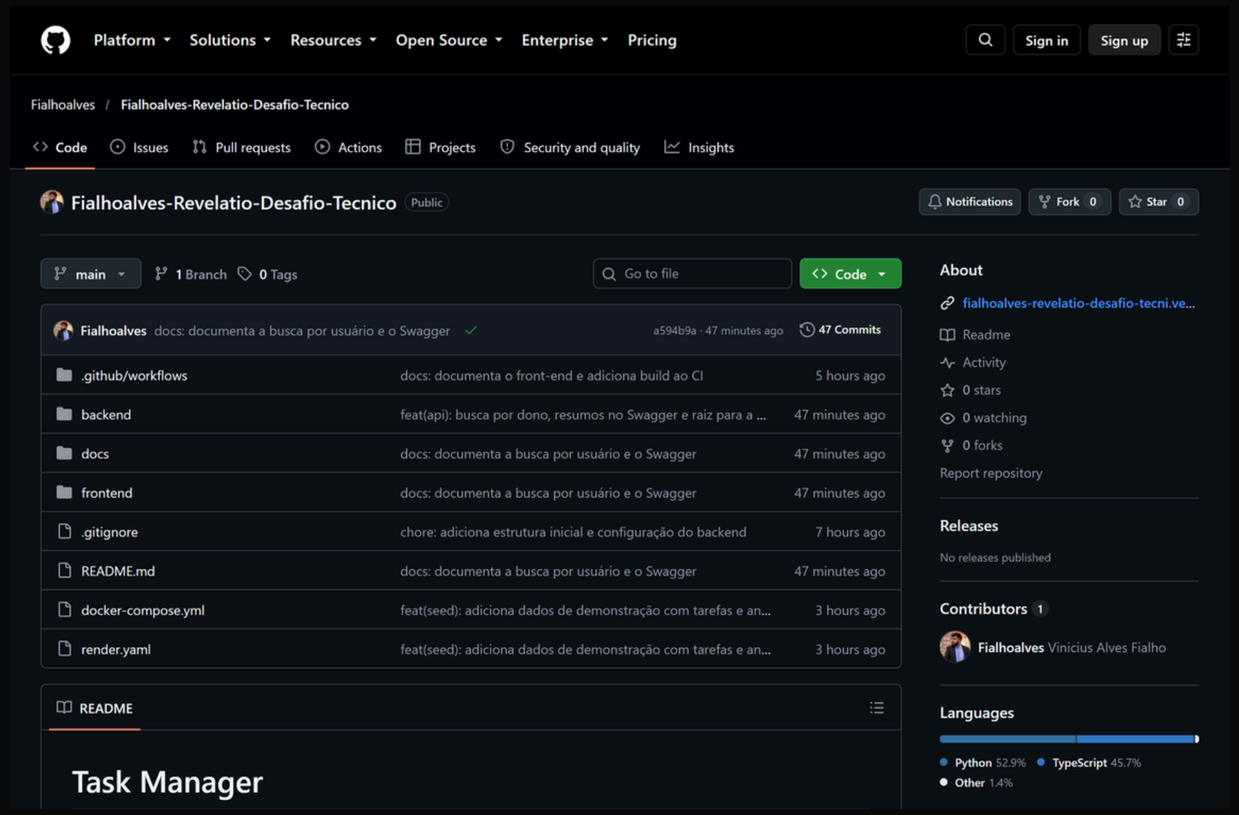
API REST e interface web para tarefas com anexos, login e dois perfis.

API e Swagger: taskmanager-api-2gkp.onrender.com

Interface: fialhoalves-revelatio-desafio-tecni.vercel.app

Back-end em primeiro lugar

Repositório público



Task Manager

README com endereços e contas de demonstração

Task Manager

API REST e interface web para organização de tarefas com anexos, autenticação e controle de acesso por perfil. Desenvolvida como desafio técnico para a vaga de Back-End da Revelatio Studio.

Demonstração

Recurso	Endereço
Interface	https://fialhoalves-revelatio-desafio-tecni.vercel.app
API e documentação (Swagger)	https://taskmanager-api-2gkp.onrender.com/docs
Vídeo de apresentação	a definir

Contas de demonstração no ambiente publicado (dados fictícios, apenas para avaliação):

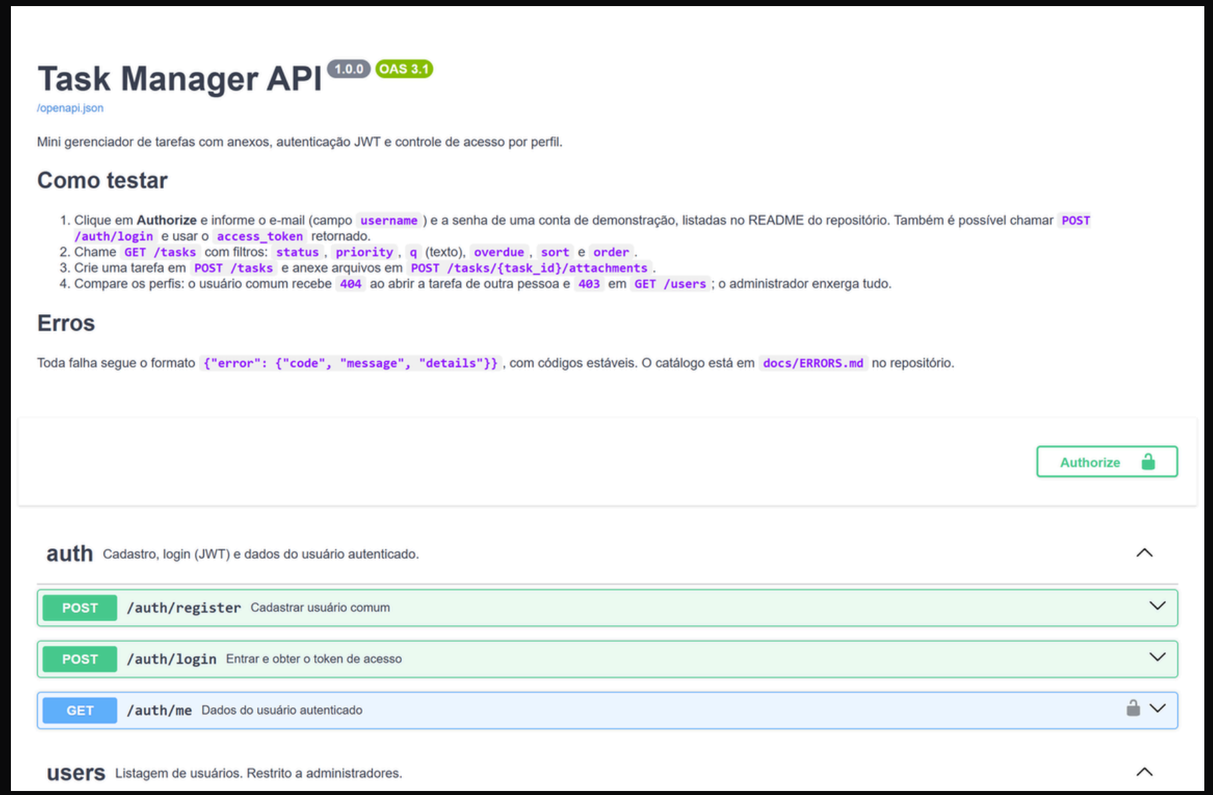
Perfil	E-mail	Senha
Administrador	admin@example.com	Revelatio-Admin-2026
Usuário comum	user@example.com	Revelatio-User-2026

O ambiente publicado já vem com dados de exemplo (tarefas em todos os status e anexos de imagem, PDF, CSV e

Abertura

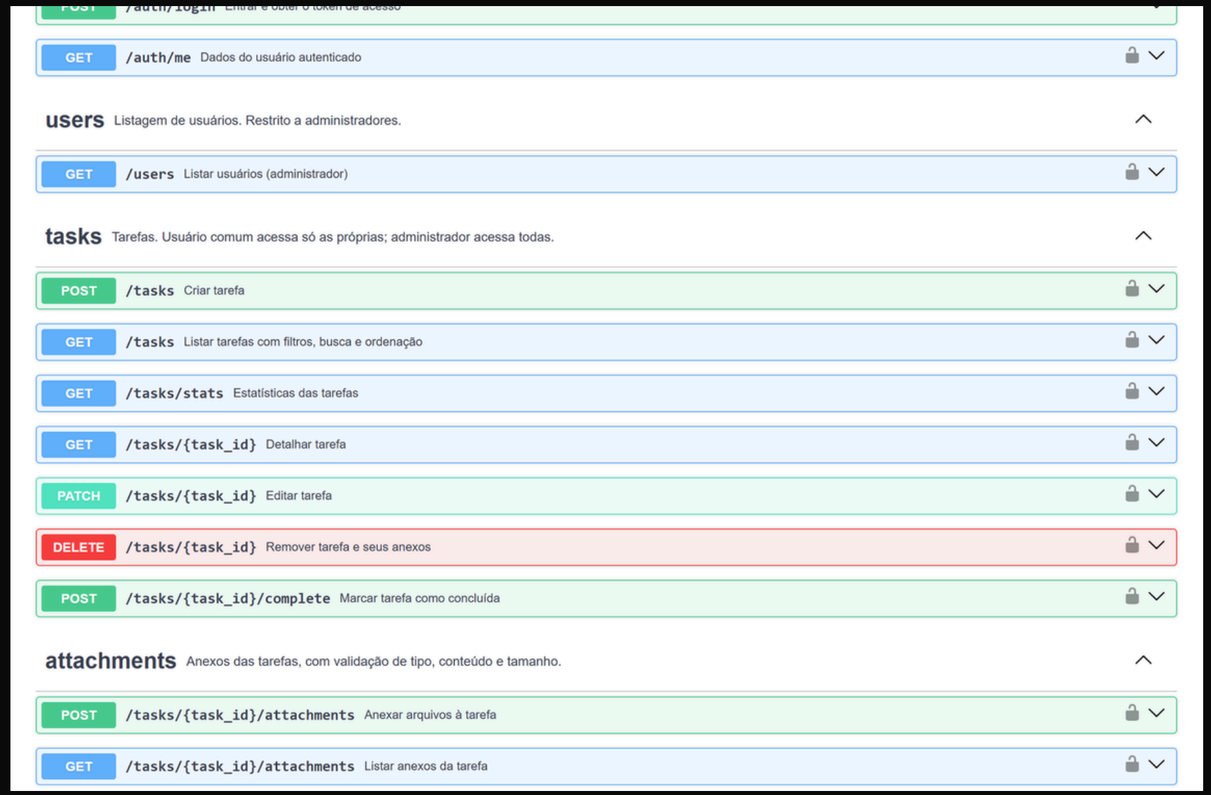
A documentação é a porta de entrada

Visão geral e roteiro "Como testar"



Task Manager

Rotas agrupadas por tag



API no Swagger

Contrato de erros

Todo erro tem o mesmo formato, com um código estável. O front decide a mensagem pelo código e nunca mostra o status.

GET /tasks/999 → 404

```
{
  "error": {
    "code": "TASK_NOT_FOUND",
    "message": "Tarefa não encontrada."
  }
}
```

Erros de validação trazem também `details`, com uma entrada por campo.

AUTH_REQUIRED	401
AUTH_INVALID_CREDENTIALS	401
PERMISSION_DENIED	403
TASK_NOT_FOUND	404
FILE_TOO_LARGE	413
FILE_CONTENT_MISMATCH	415
VALIDATION_ERROR	422
RATE_LIMITED	429

17 códigos documentados

Catálogo em docs/ERRORS.md, com a ação sugerida no front para cada um.

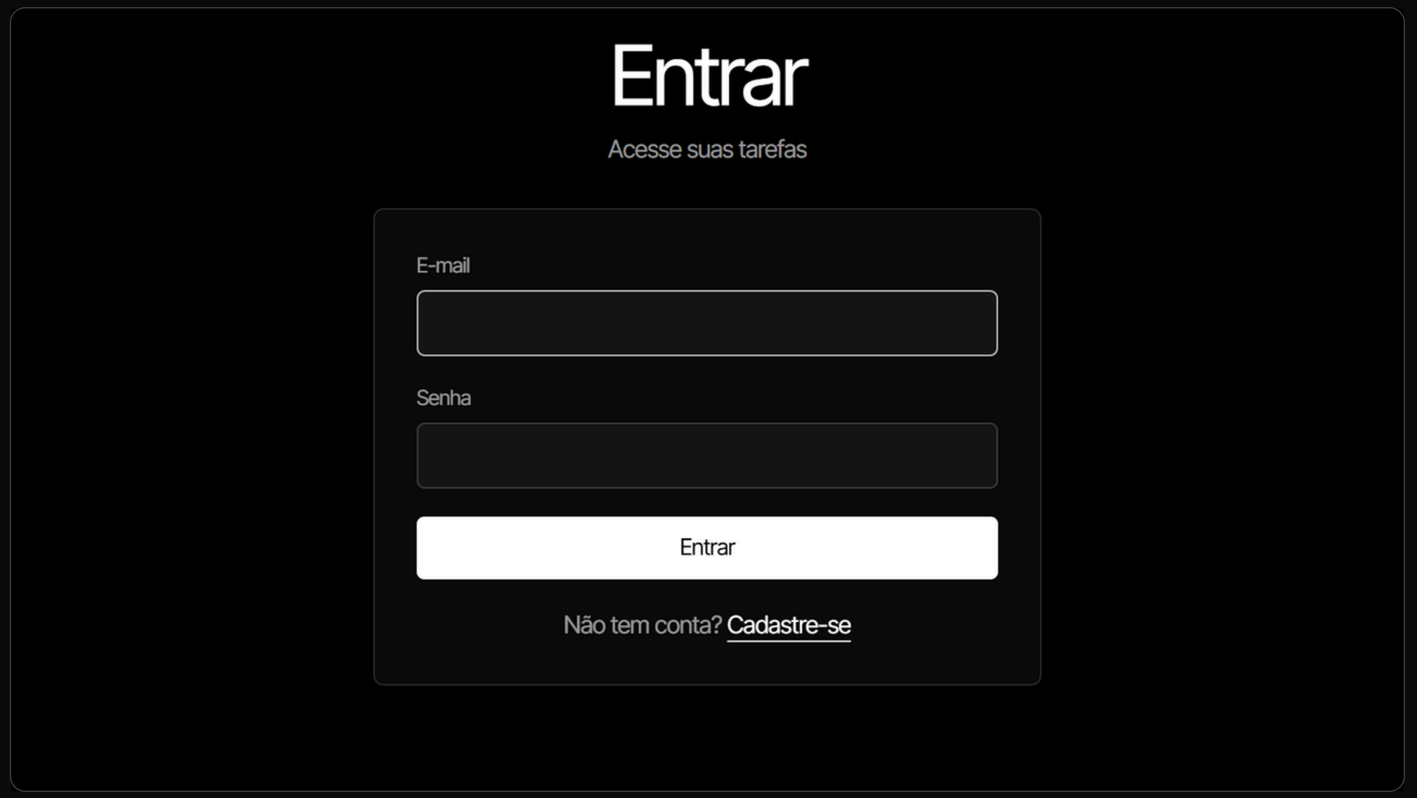
Erro inesperado não vaza detalhes

O servidor registra o erro no log e responde apenas INTERNAL_ERROR.

Um teste por código

Cada código do catálogo tem status e código verificados em teste.

Login devolve um JWT



Validação de anexos

O servidor não confia no tipo informado pelo cliente.

1

Tipo permitido

Lista de tipos aceitos: imagens, PDF, TXT, CSV, Word e Excel

415

`FILE_TYPE_NOT_ALLOWED`

2

Assinatura confere

O tipo declarado vem do cliente: os primeiros bytes são comparados ao formato

415

`FILE_CONTENT_MISMATCH`

3

Tamanho até 10 MB

O limite é aplicado durante a gravação, em blocos

413

`FILE_TOO_LARGE`

4

Nome em UUID

O nome original fica só no banco; o arquivo nunca usa o nome enviado

5

Armazenamento

Disco ou PostgreSQL, pela mesma interface: save, read, delete

Download só pela API

A permissão é conferida a cada pedido. Não existe link público para o arquivo.

Cabeçalhos de segurança

```
X-Content-Type-Options: nosniff
Content-Security-Policy: sandbox
```

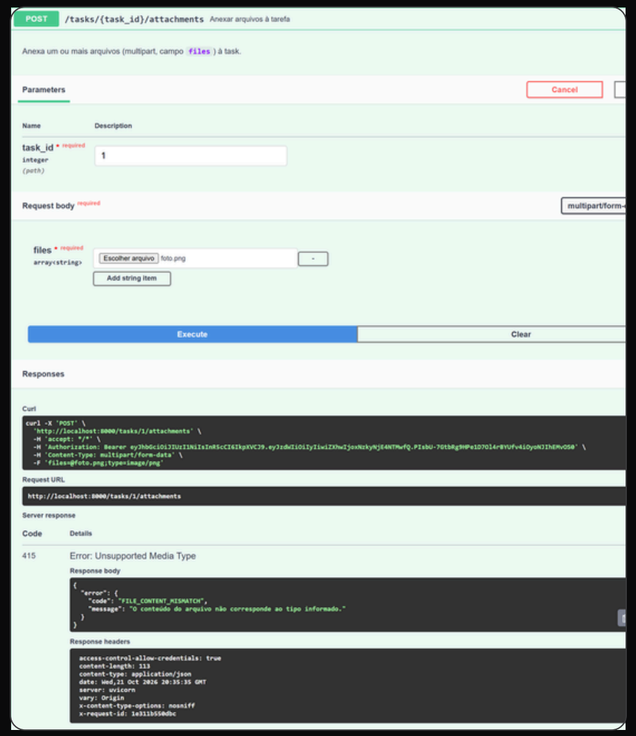
Mesma regra de acesso da tarefa

Quem não vê a tarefa recebe 404 também no anexo.

Arquivo falso: 415

Por quê: o tipo declarado vem do cliente. Um texto salvo como foto.png declara image/png, mas os primeiros bytes não batem com a assinatura do PNG.

Requisição: foto.png



Resposta

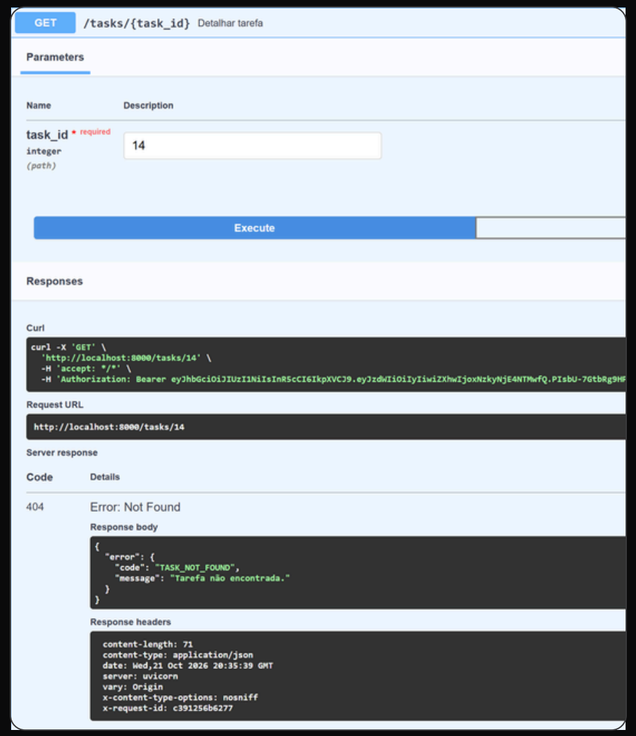


Task Manager

API no Swagger

Tarefa de outra pessoa: 404

Requisição: task_id 14



Resposta



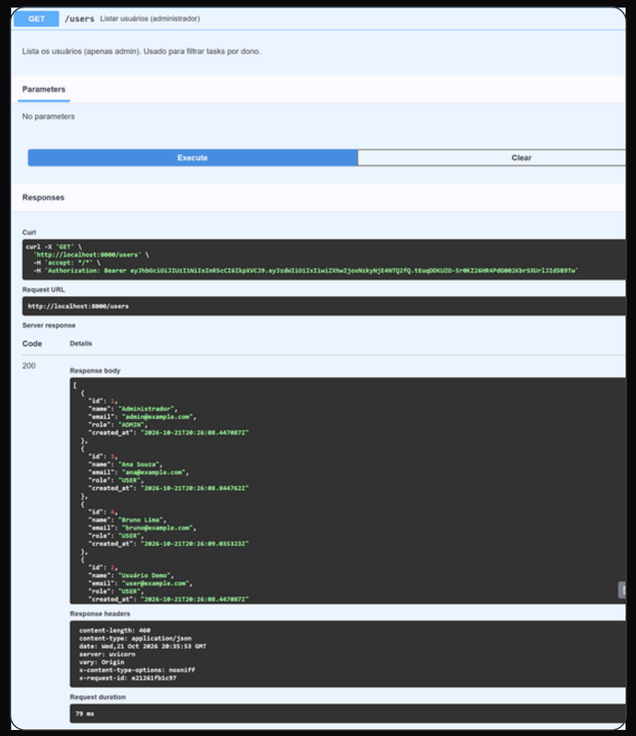
Task Manager

API no Swagger

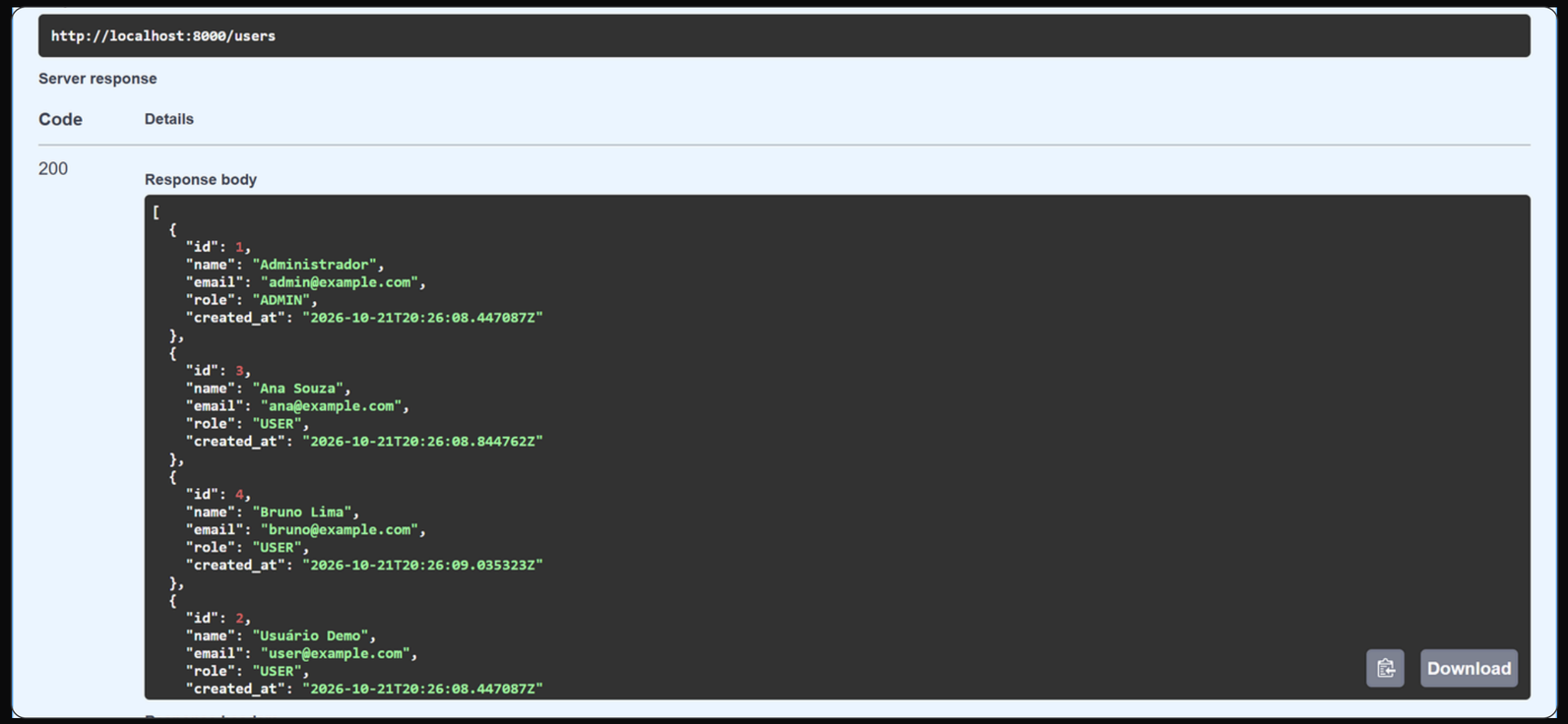
Administrador: usuários

Por quê: a chamada que deu 403 para o usuário comum devolve os quatro usuários para o administrador. Quem decide é o servidor, pelo perfil.

Requisição: administrador



Resposta



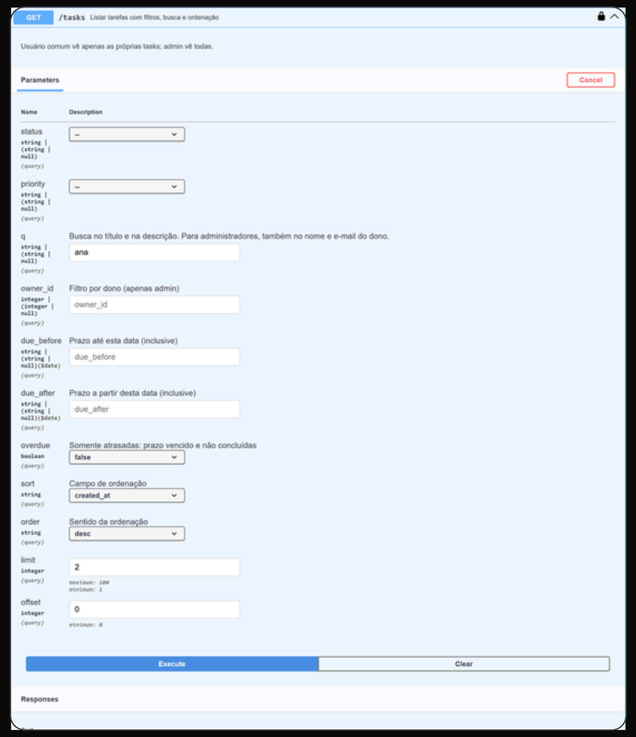
Task Manager

API no Swagger

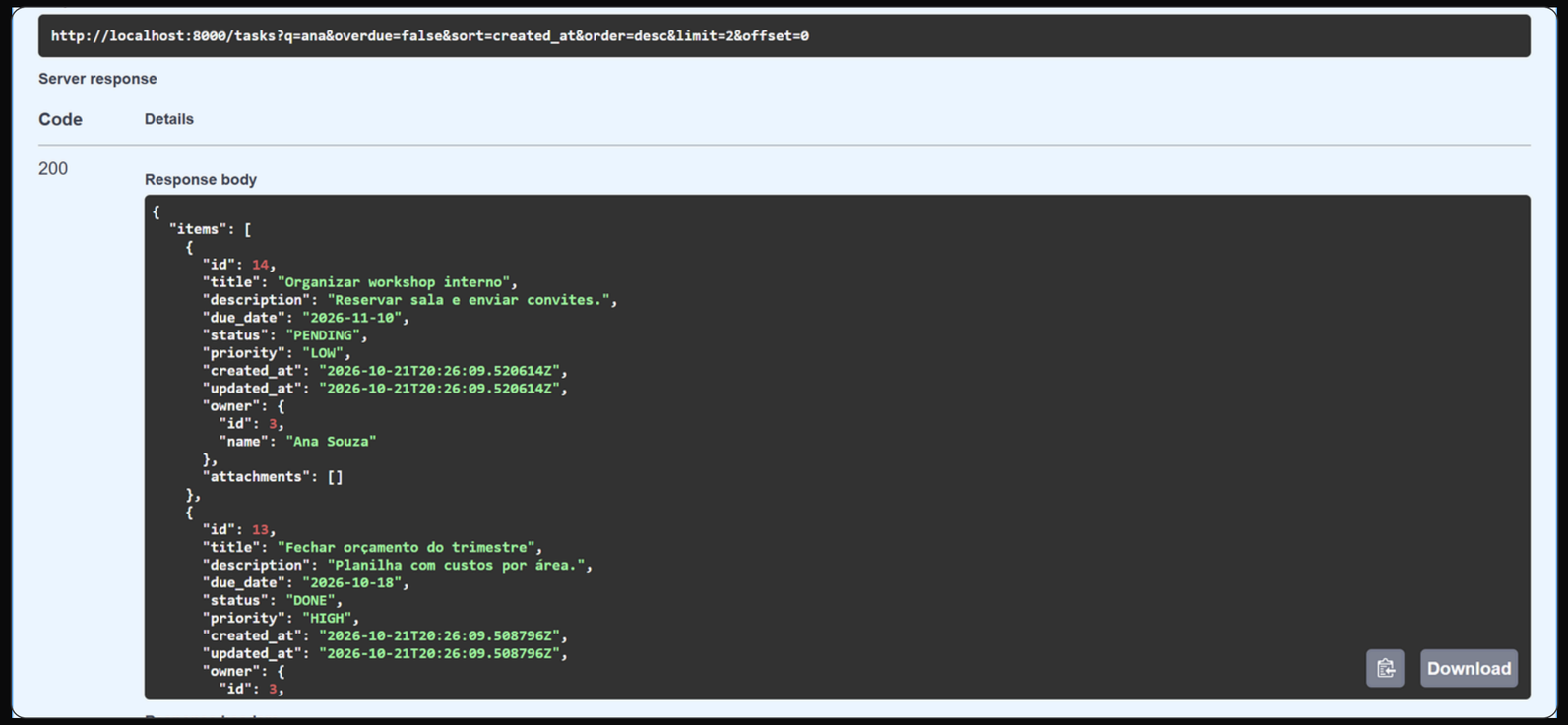
Administrador: busca por dono

Por quê: para o administrador, o parâmetro q também procura no nome e no e-mail do dono. O usuário comum busca só nas próprias tarefas.

Requisição: q=ana



Resposta

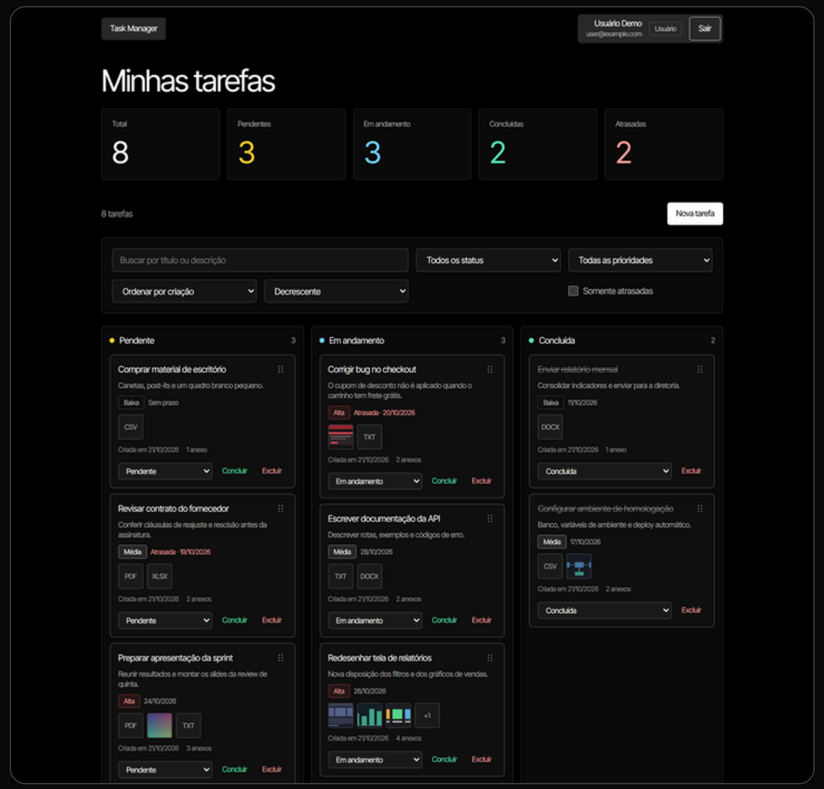


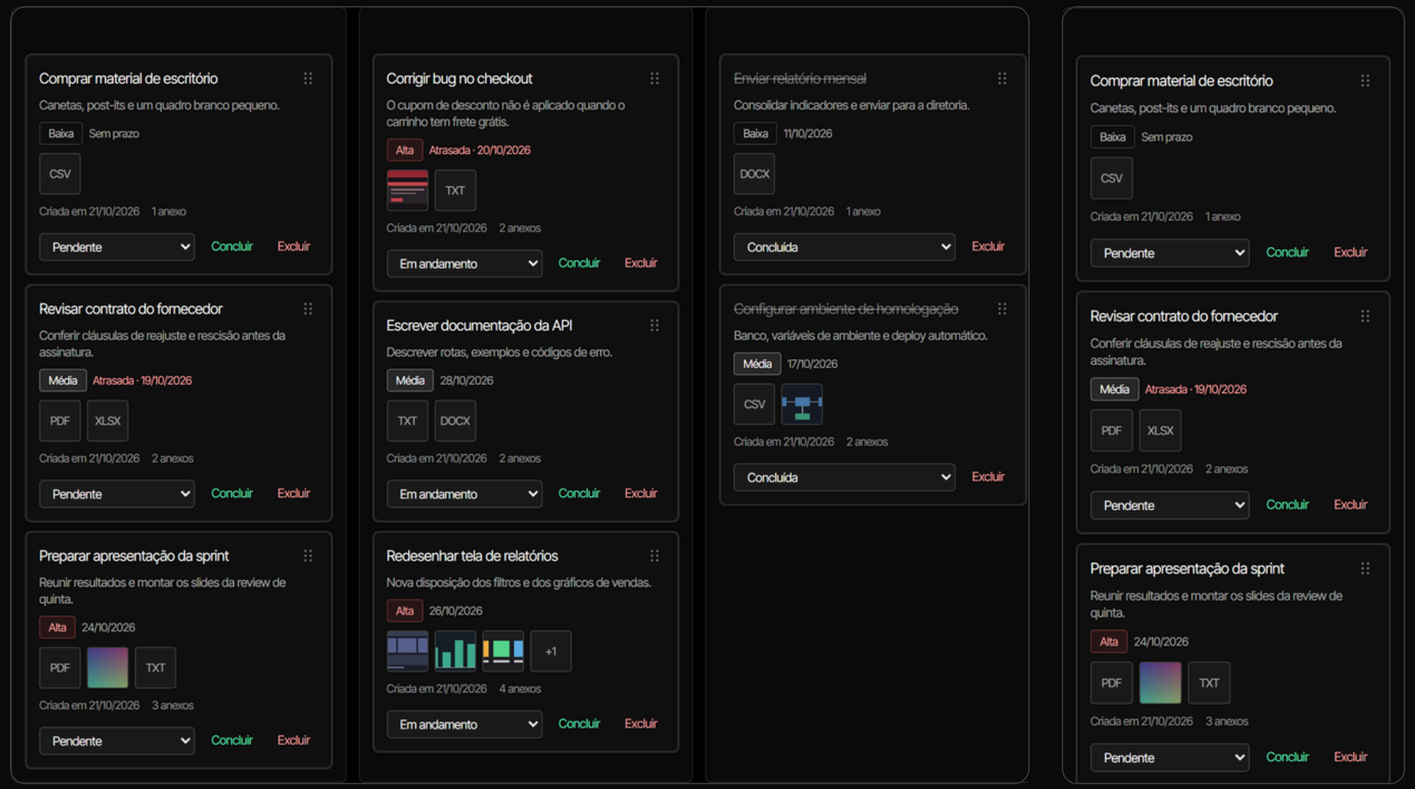
Task Manager

API no Swagger

A interface é um cliente da API

Por quê: o enunciado pede o front só para testar. Ele guarda o token, chama a API e mostra o que ela devolve; nenhuma regra de acesso mora nele.



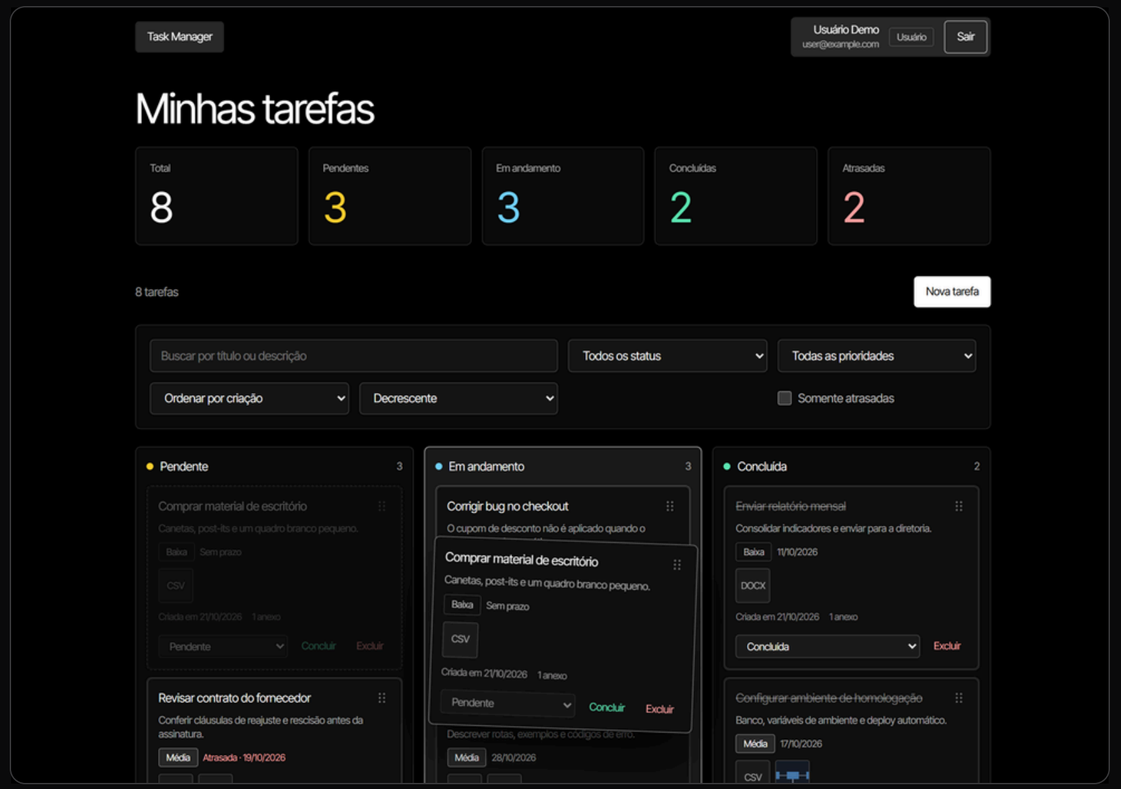


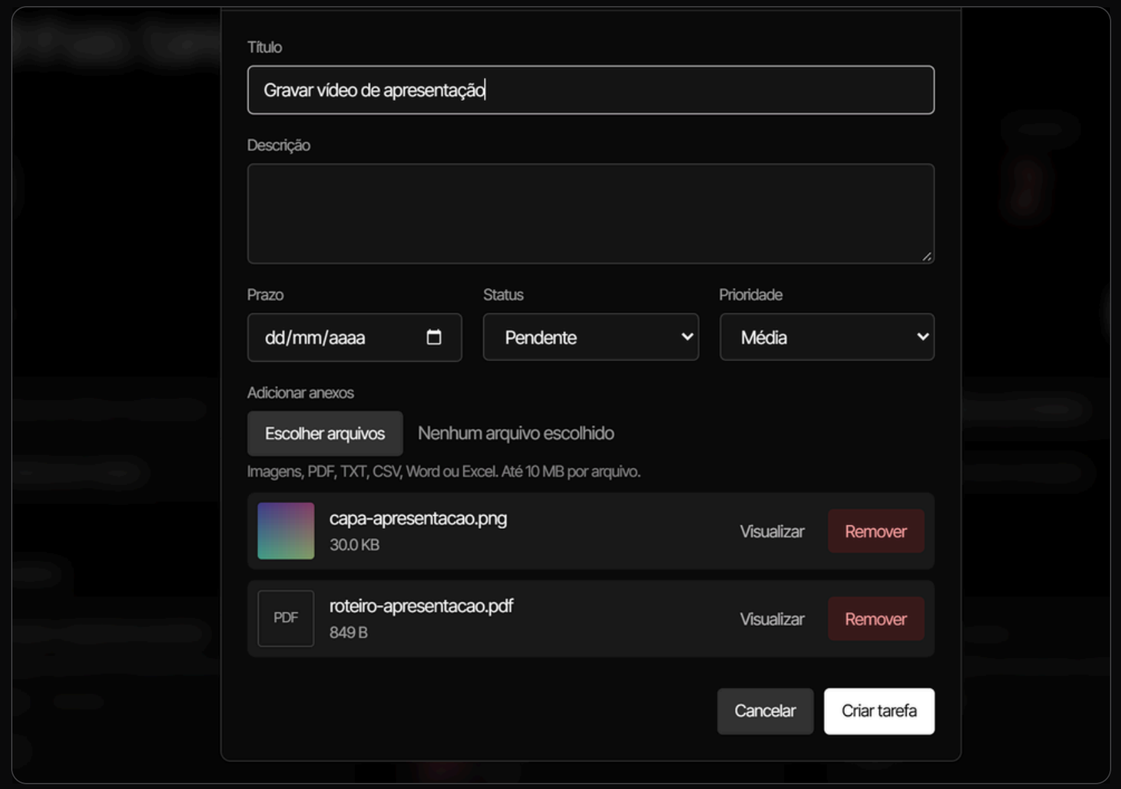
Task Manager

Interface

Arrastar um cartão chama o PATCH

Por quê: a fonte da verdade é a API; se ela recusar, a tela desfaz o movimento. A prévia dos arquivos é conveniência, quem valida é o servidor.





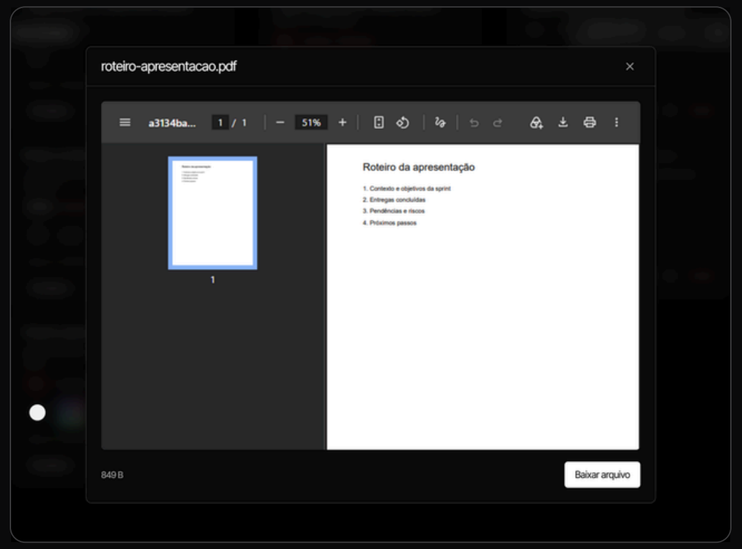
Task Manager

Interface

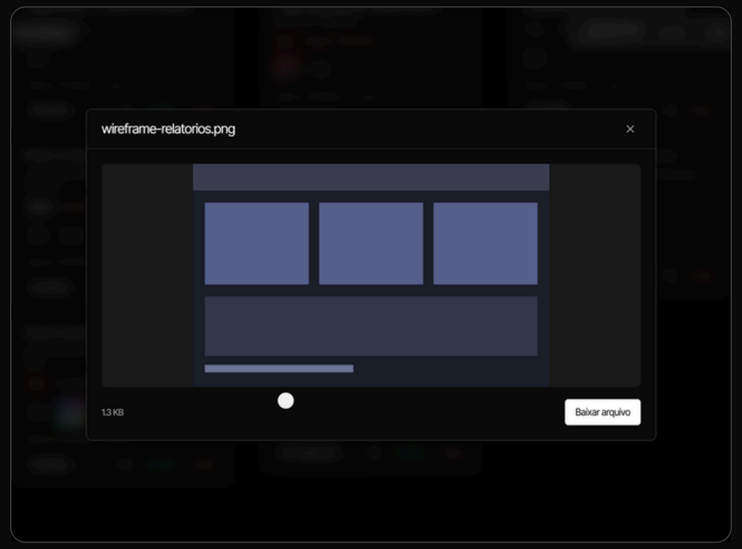
Preview de anexos no cartão

Por quê: os arquivos só chegam pela API, com checagem de permissão. PDF, imagem e texto abrem sem sair do quadro.

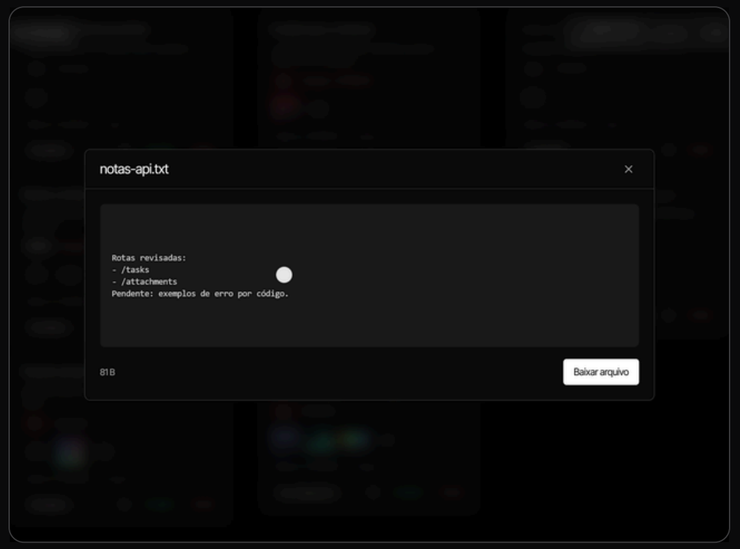
PDF



Imagem



Texto

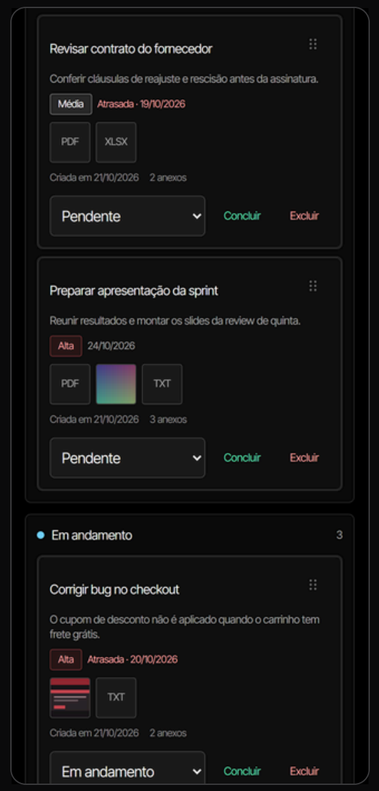


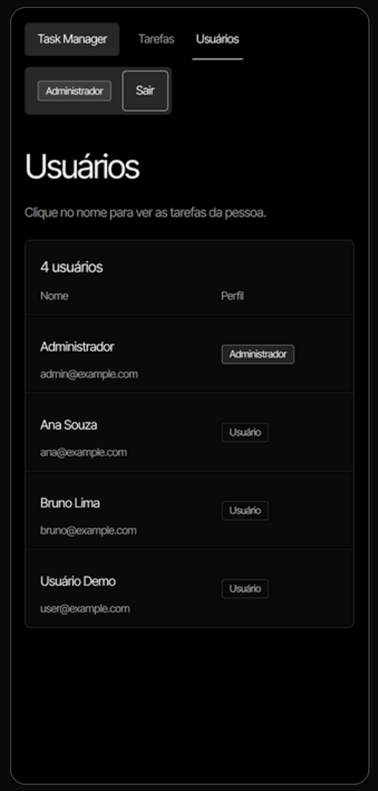
Também no celular

Mesmo cliente e mesma API, com layout para telas de 390 px.



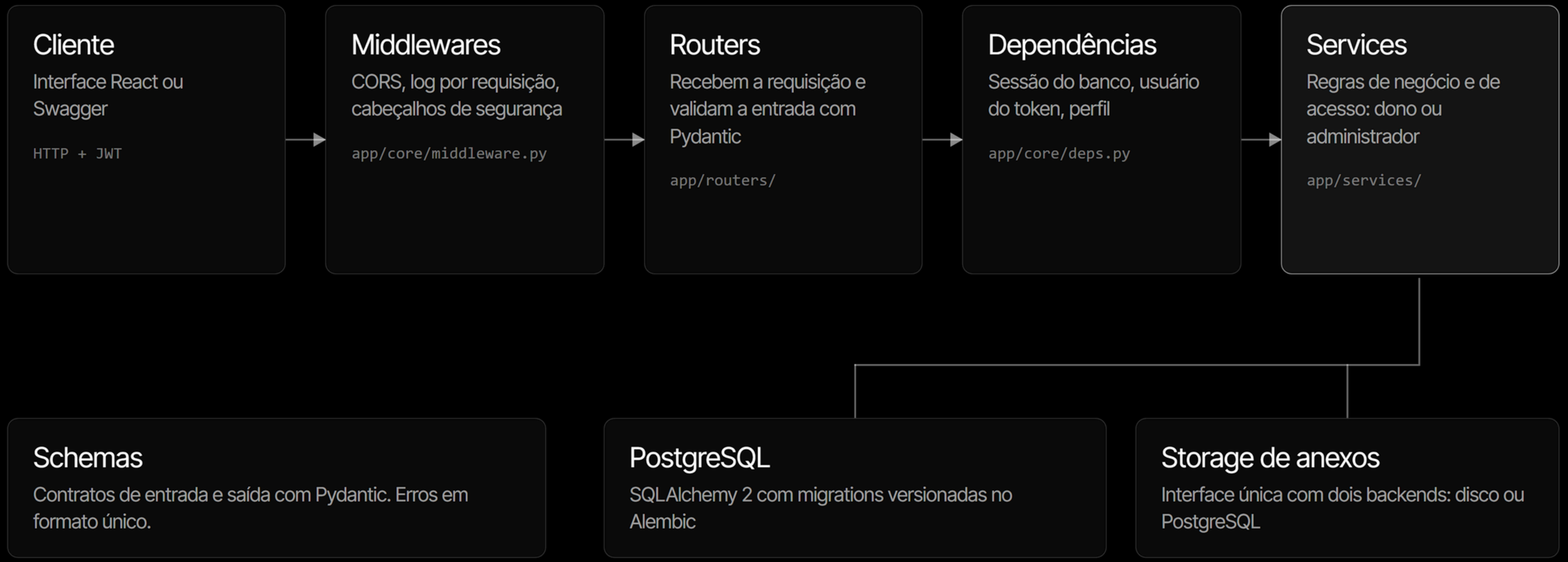






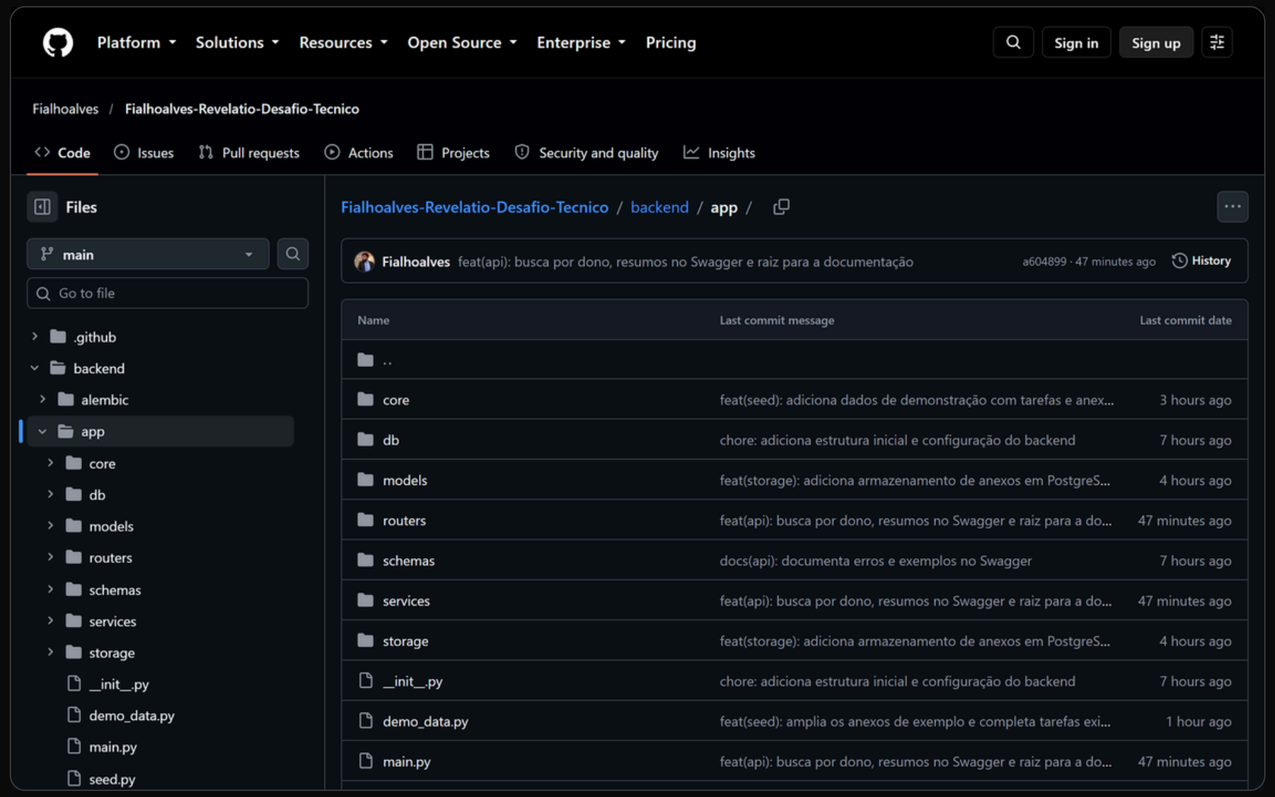
Arquitetura em camadas

O router só recebe e valida. A regra de negócio e a de acesso ficam nos services.



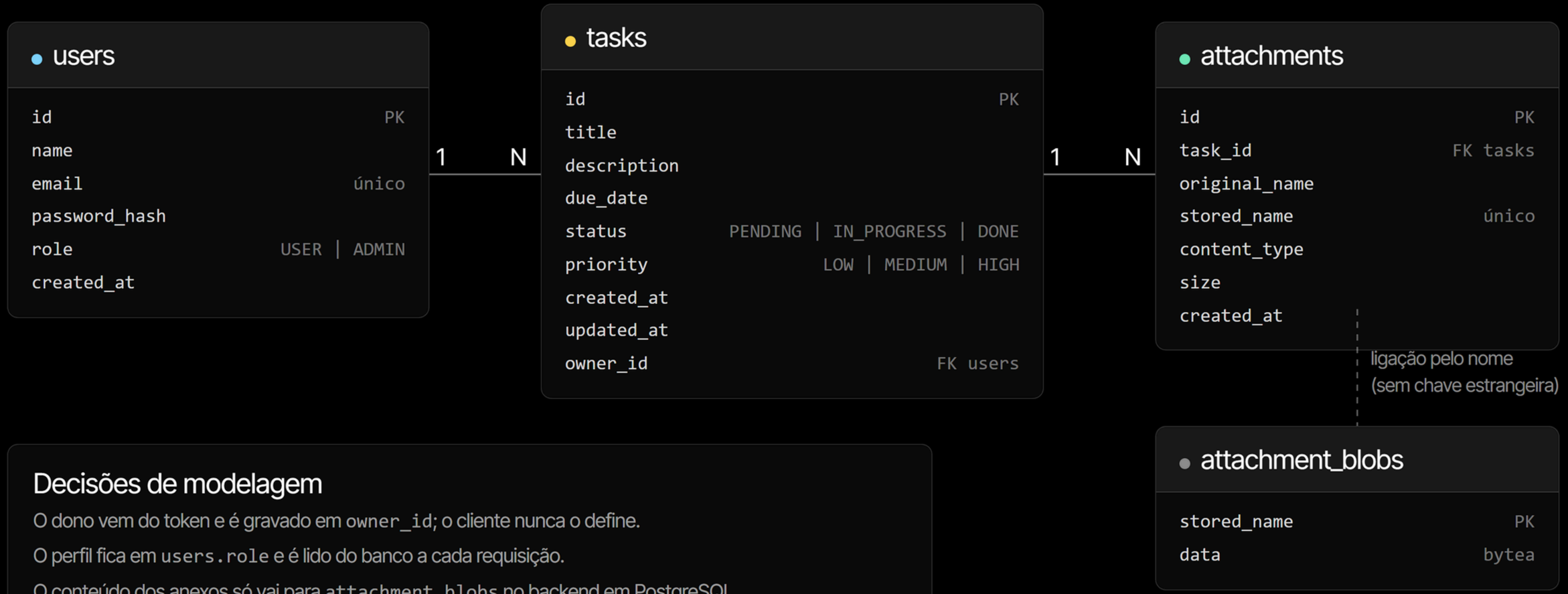
Uma pasta por responsabilidade

Por quê: routers, services, models, schemas, storage e core espelham as camadas do diagrama. Cada regra tem um endereço previsível.



Modelo de dados

Três tabelas principais e uma auxiliar. Exclusão em cascata de tarefa para anexos.



Decisões de modelagem

O dono vem do token e é gravado em `owner_id`; o cliente nunca o define.

O perfil fica em `users.role` e é lido do banco a cada requisição.

O conteúdo dos anexos só vai para `attachment_blobs` no backend em PostgreSQL.

Deploy e entrega

A API e o banco no Render, o front na Vercel, e o CI no GitHub Actions a cada push.

Navegador
Interface React ou Swagger

→

Vercel
Front-end React, Vite e Tailwind

→

Render
API FastAPI em Docker, com migrations e dados de exemplo na inicialização

→

PostgreSQL
Dados e anexos

GitHub
Repositório público. Cada push publica o front e a API.

CI: GitHub Actions
pytest em PostgreSQL 16, migrations aplicadas e revertidas, build do front

Docker Compose
API e banco locais com um comando

O plano que mudou
O deploy previsto era na Railway, com volume para os anexos. Ela ficou indisponível na véspera da entrega. No Render o disco gratuito é apagado a cada reinício, então os anexos passaram a ser guardados no PostgreSQL, escolhidos por variável de ambiente.

Testes automatizados

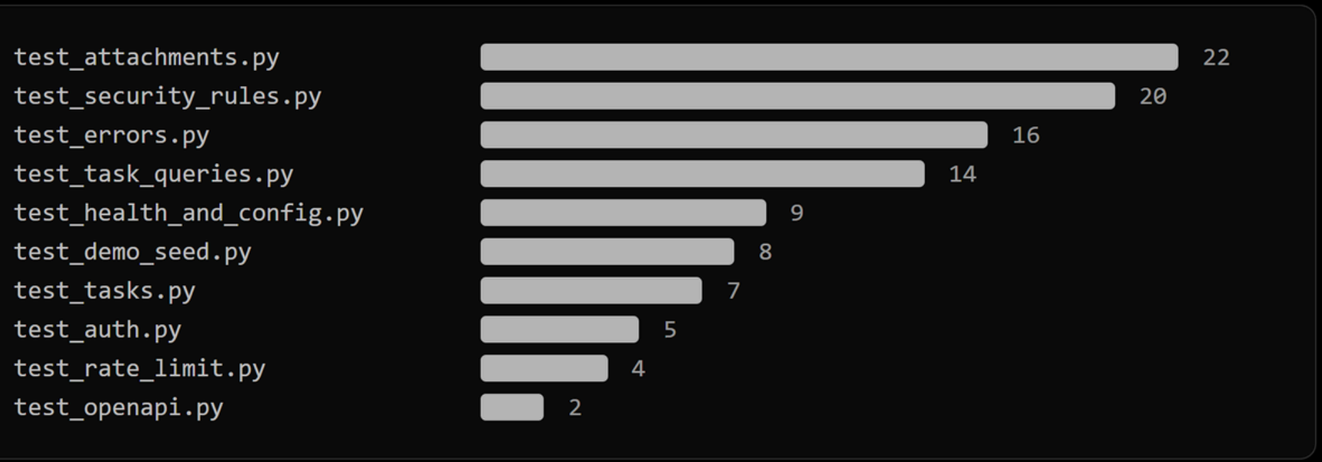
A suíte exercita a API por HTTP, passando por autenticação, validação, regras de acesso e banco.

107
execuções

78
cenários distintos

17
rotas verificadas contra login

2
bancos: SQLite e PostgreSQL 16



Quebra de propósito

Ao remover a checagem de dono, 4 execuções falham (3 cenários). Isso mostra que os testes protegem a regra de acesso.

Guarda de rotas

Um teste lê o schema OpenAPI e falha se alguma rota nova ficar sem autenticação.

Suíte completa: 107 passed

Por quê: os testes chamam a API por HTTP, passando por autenticação, validação, regras de acesso e banco, como um cliente real.

backend

```
$ pytest -q

.....................................................................  [ 67%]
.....................................                                  [100%]
107 passed in 63.14s (0:01:03)
```

107 execuções, 78 cenários

Por quê a diferença: o teste de autenticação roda uma vez para cada rota, e os de anexos rodam nos dois backends de armazenamento.

backend

```
$ pytest --collect-only -q   # execuções por arquivo

22 tests/test_attachments.py
5 tests/test_auth.py
8 tests/test_demo_seed.py
16 tests/test_errors.py
9 tests/test_health_and_config.py
2 tests/test_openapi.py
4 tests/test_rate_limit.py
20 tests/test_security_rules.py
14 tests/test_task_queries.py
7 tests/test_tasks.py
```

Quebra de propósito

Por quê: um teste só vale se falhar quando a regra quebra. Sem a checagem de dono em get_accessible_task, 4 execuções falham em tarefas, erros e anexos.

backend

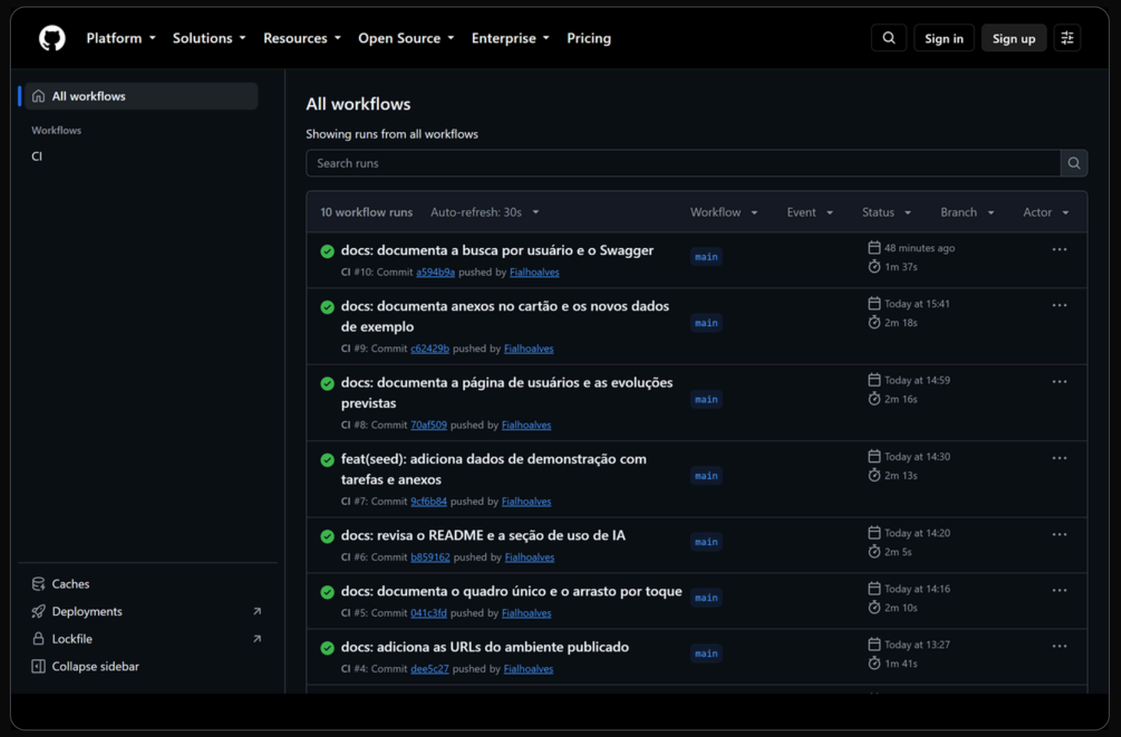
```
$ pytest -q tests/test_tasks.py tests/test_errors.py tests/test_attachments.py   # sem a checagem de dono

FAILED tests/test_tasks.py::test_user_cannot_touch_others_task
FAILED tests/test_errors.py::test_other_users_task_looks_like_not_found
FAILED tests/test_attachments.py::test_attachment_access_follows_task_owner[local]
FAILED tests/test_attachments.py::test_attachment_access_follows_task_owner[database]
4 failed, 41 passed, 1 warning in 29.80s
```

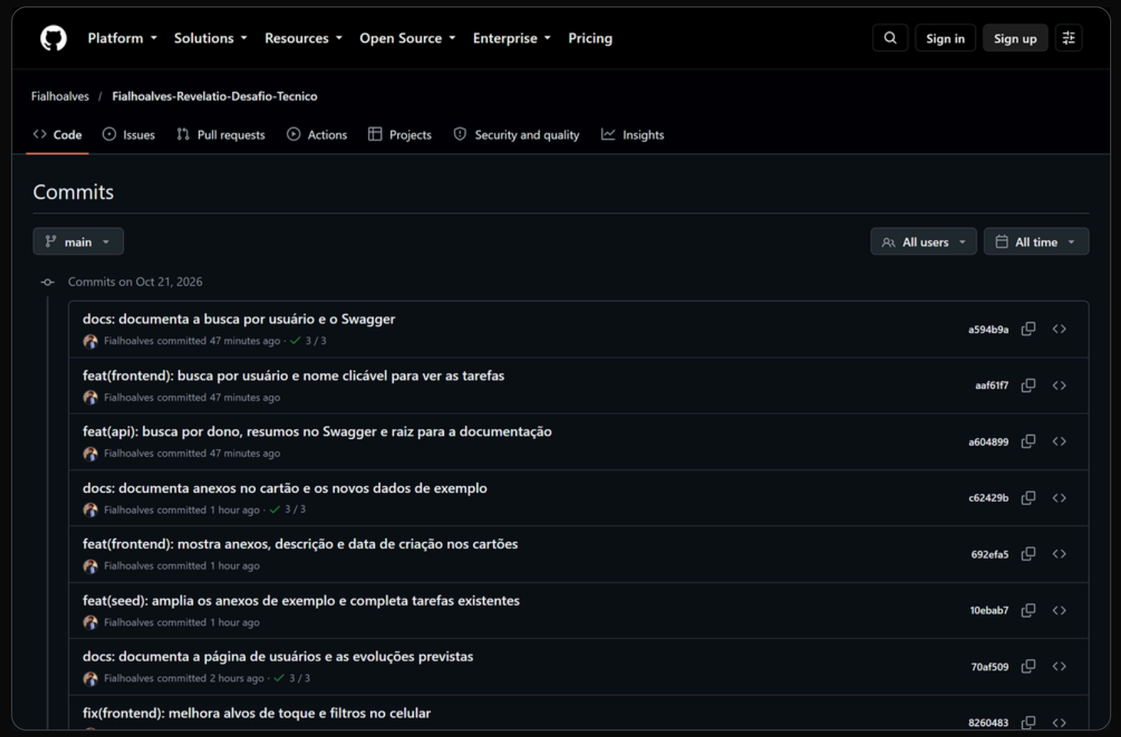
CI e histórico

Por quê: cada push roda os testes contra PostgreSQL 16, aplica e reverte as migrations e compila o front. Um erro aparece antes do deploy, não em produção.

GitHub Actions: execuções verdes



Commits em Conventional Commits



Uso de IA

Ferramenta

Claude Code, como assistente

Estrutura do backend, testes, migrations, Docker, CI, documentação e interface em React.

Autor

Decidido e conferido

Stack, regras de acesso, contrato de erros e escopo. Cada entrega executada, testada e conferida no navegador.

Por quê conferir: a revisão por execução achou um teste de autenticação por rota que não rodava nenhum caso, por uma mudança no framework.

Limitações e próximos passos

Limitações conhecidas

- Limite de login por processo
- Sem refresh token
- Arquivos sem varredura de malware
- Front sem testes automatizados

Com mais tempo

- Refresh token
- Armazenamento de objetos
- Testes do front